

Review of DNABERT-2 Architecture and Improvements for DNA Sequence Error Correction

Luke Hendrickson* Adam Camerer*

* Department of Computer Science, Missouri University of Science and Technology, Rolla, MO, USA { [ldhtnp](mailto:ldhtnp@mst.edu), [ajc3xc](mailto:ajc3xc@mst.edu) }@mst.edu

ABSTRACT

This paper provides a review of the DNABERT-2 architecture, highlighting advancements over the original DNABERT model for DNA sequence error correction. By leveraging innovations in tokenization, attention mechanisms, and multi-species training datasets, DNABERT-2 achieves improved performance and efficiency across diverse genomic tasks. We detail the architectural modifications, dataset preparation, and experimental results from fine-tuning DNABERT-2 for DNA sequence error detection, showcasing significant gains in accuracy and recall. Our findings underscore the potential of genomic language models to enhance sequencing accuracy and reliability while identifying key areas for future improvements in genomic error correction pipelines.

1 INTRODUCTION

Genomics has been rapidly evolving over the past few decades. The advent of Next Generation Sequencing (NGS) has led to exponential growth in sequencing power and decline in costs for sequencing, and new third generation sequencing technologies can read much longer sequences[3]. A major issue facing sequencing is sequencing errors, in which a certain percentage of the sequence does not match the true sequence. Genomic Language Models (gLMs), trained on pre-existing genomic data can help detect and correct errors as well as improving read alignment and improving sequencing in repetitive regions[4]. Our project goal is to improve gLM error detection capabilities, a task critical to high throughput but low accuracy data. Below we will explain the evolution of DNA Sequencing and gLMs, as well as the current breakthroughs and challenges for each.

1.1 DNA Sequencing

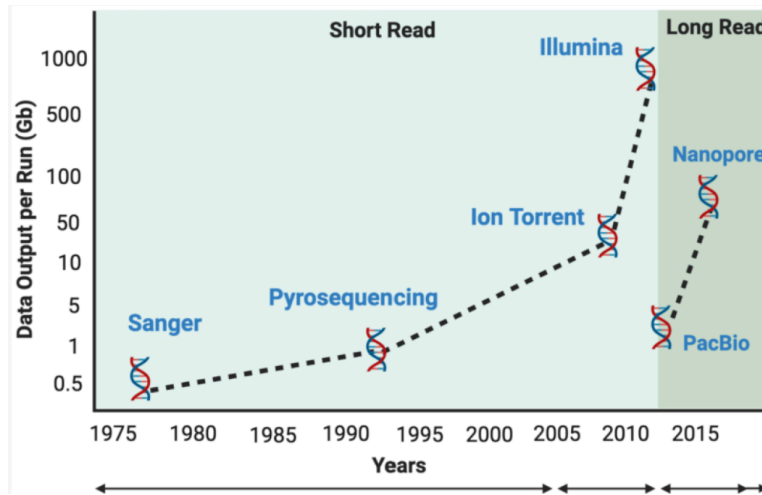
The DNA Sequencing revolution over the past 3 decades has many similarities to the digital revolution. Like the transistor being invented in 1947, the double helix strand of DNA was discovered in 1947 [3]. Gilbert and Maxam developed a DNA sequencing method from chemical degradation using radioactive phosphate and X-ray films in 1964, but the radioactive phosphate, cutting chemicals and under 500 base pairs of sequencing made the process unsafe and not

scalable enough. [1,2,3]. In 1974, Sanger et al. developed an approach using chain elongation termination, fluorescent ddNTP (dideoxynucleoside triphosphate) and capillary electrophoresis to identify DNA [1,2,3]. This led to a breakthrough in DNA sequencing technology, and in 1987 the first commercial sequencing machine, the ABI 370 was built [3]. Like computers, giant sequencing machines would take up entire buildings, and were extremely costly and slow at sequencing. Sanger inspired machines became automated and more efficient to process hundreds of samples at a time, but were still incredibly slow and could not only read short shreds of DNA [2]. The Human Genome Project, a research initiative to map the entirety of the human genome took 13 years and 3 billion dollars, and largely used Sanger based machines to sequence the DNA [2].

The long sequence time and extreme cost demonstrated a demonstrable need for machines that could process more data at a smaller timescale with less cost, creating the incentives needed to develop second generation sequencing machines which massively parallelized genome sequencing. These second generations relied on sequencing by synthesis and fragmentation / amplification of DNA so it could be cut into small enough chunks to process in parallel for 1000 to millions of base pairs. The first iteration of second generation machines was the Roche 454 developed in 2005, which used pyrosequencing and emulsion PCR as its basis, and worked for around 400-1000 base pairs read lengths at about 1 million base pairs per run[1,2,3]. Ion Torrent, another machine relied on Emulsion PCR and micro well chips detecting the release of hydrogen ions that had 1 million wells per chip to allow for massive parallelization, but only could work with 200-400 base pairs [2,3]. Illumina came next in 2006, which used Bridge amplification, reversible dye terminators and polymerase sequencing by synthesis to sequence 40 million pairs in 3-4 days and 36-300 read length with only a 0.1% error rate and was the gold standard of sequencing [1,3]. Following this in 2007 came SOLiD (Sequencing by Oligonucleotide Ligation and Detection) used emulsion PCR and fluorescence like Roche 454, but used ligase sequencing of amplified fragments, achieving relatively similar sequencing rates to Illumina and a .15% error rate, but had a short read limit of 75 base pairs [1,2,3]. With the rise of new high throughput sequencers, DNA sequencing became 4 orders of magnitude less expensive between 2007 and 2012[1] (that's absolutely mind boggling). Suddenly, DNA sequencing became within the reach of hospitals and midsize organizations, similar to how the personal computer went from mainframes taking up an entire room to becoming an expensive, but common appliance fitting onto a desk in the 1980s and 1990s.

While 2nd generation sequencers continue to be developed and improved to this day, they were limited by their ability to only work with short reads and relying on amplification and transcription of the original DNA strand, causing potential inaccuracies when trying to piece together a long sequence of DNA. 3rd generation sequencers revolutionized this by being capable of long read sequencing of lengths 10,000 to 30,000 on average, improved throughput and directly sequenced the DNA in real time [2,3]. In 2010, Pacific Biosystems (PacBio) using a

Single Molecule Real Time (SMRT) method and Zero Mode Wavelength Technology in which thousands of ZMW well with polymerase allow the DNA to be directly sequenced, using parallelized fluorescence in a similar manner to 2nd generation systems and a laser to correspond fluorescence to the DNA base pair, and achieved a considerably faster throughput of 100 million bases in just 8 hours with a 99.9 percent accuracy to date with the Sequel 2e system [1,2,3]. In 2014 Oxford Nanopore based sequencing was launched with the portable MinION at only \$1000, a record low price point [1]. Oxford Nanopore was based off of nanopore technology developed in the early 1990s, and relies on a passing a single stranded DNA through natural protein or synthetic nanopore with electric current running through it and measuring the disruption, and achieved a throughput in only a few hours with an 87 to 98 percent accuracy rate [1,2,3]. Instead of relying on a photocopier size sequencer costing hundreds of thousands to millions of dollars, it is now feasible for a consumer or a small lab to afford purchasing sequencing equipment that can sequence in a reasonable amount of time, albeit with worse accuracy than an expensive machine. This influx of faster, higher throughput but more error prone (especially Oxford Nanopore) long read data has created a growing need for methods to detect and correct sequencing errors.

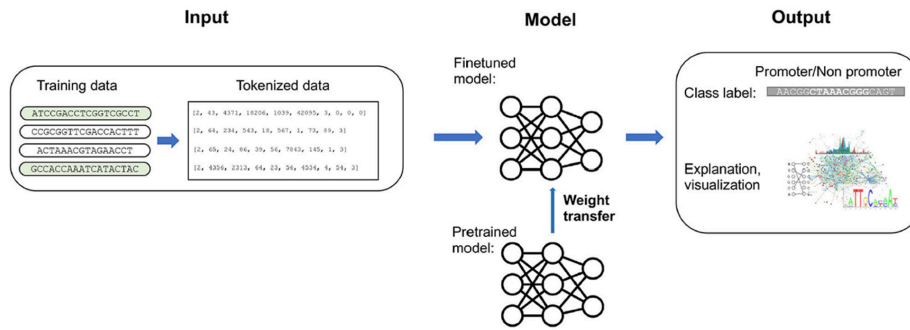


1.2 Genomic Language Models (gLMs)

As genomic data became affordable in the 2010s, the AI boom was starting to take off. With the exponentially decreasing cost of genomics data and reductions in sequencing errors of data, AI models have increasingly vast repositories of genomic data to train on [1]. Initially, CNNs and LSTMs were primarily used in the development of Genomic Language Models, but transformers have become increasingly popular from 2020 onwards due to their ability to look at all the layers of a sequence (MLM) or before a certain sequence (CLM), as well as being able to be fine tuned for very specific tasks. Masked Language Modeling (MLM) typically masks a certain percentage of the input from the sequence randomly (usually 15%), and then tries to predict for that missing sequence, whereas Casual Language Modeling (CLM), which models like ChatGPT use, predicts the next token using the preceding tokens [4].

MLMs and CLMs originated from Natural Language Processing (NLP), where transformer based approaches of training on vast sets on unlabelled data to “fill in the blank” with MLMs or guessing the rest of the sentence with CLMs experienced remarkable success [4]. Basing themselves off of NLP architecture, Protein language models (pLMs) had remarkable success by using vast repositories of unlabeled data from sources such as UniProt’s protein sequence database for structure prediction and variant effect prediction, so training on DNA was a natural next step [4]. Translating into genomics has proven more difficult than proteins, due to the decreased diversity of genomes compared to the hundreds of millions of protein sequences, in addition to duplicated sequences (over 50% of the human genome is repetitive), alongside the enormous size of eukaryotic genomes such as humans and plants that makes it difficult for transformers to process them, as they scale in runtime and memory usage quadratically, limiting input sequence sizes[4]. SSMs (state space models) such as HyenaDNA and Mamba scale linearly with sequence length, with HyenaDNA being able to support inputs up to 1 million BP, and Transformer SSM combinations such as EVO being able to take 8kb long sequences for training and 131kb for transfer learning [4]

Genomic Language Models have multifaceted uses, including generating synthetic sequences, predicting functional constraints such as learning transcription factor binding site (TFBS) motifs, single nucleotide polymorphisms (SNPs), insertion / deletions, variant impact and error detection among many other things, allowing us to better understand the genomic language [4]. While current gLMs are powerful tools, there is much work to be done to improve them. Early gLMs were originally direct NLP adaptations, and as models evolve they will increasingly use genomic context to improve their predicting ability and utility for a wide array of genomic tasks.

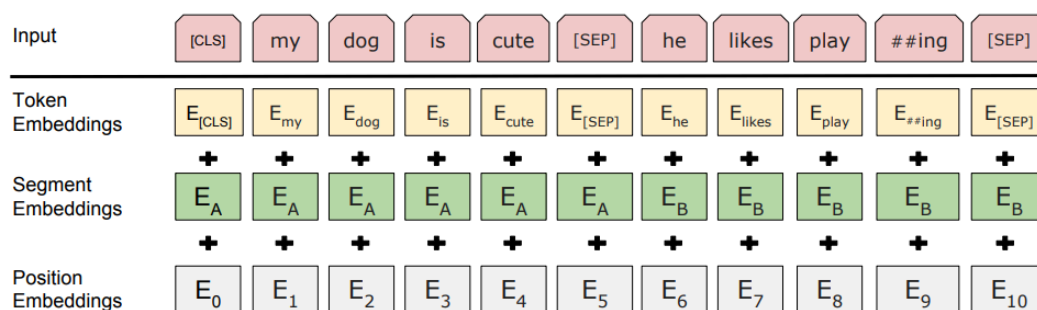


2 BACKGROUND

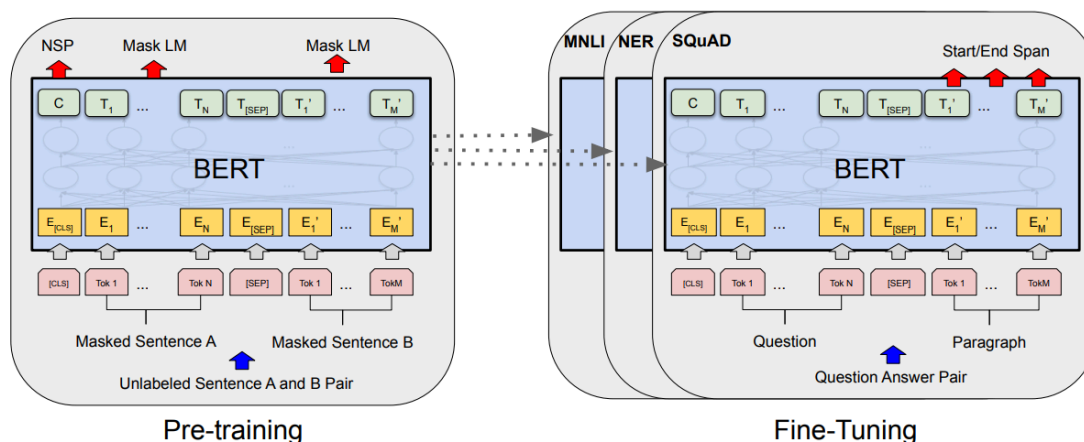
Transformer architectures, originally designed for NLP have been remarkably useful predicting the sequences and structures of genomic data. BERT, a MLM based transformer for NLP was used as the basis for DNABERT, an innovative model to predict genomics data and important functions that DNABERT is based off of.

2.1 Bidirectional Encoder Representations from Transformers (BERT)

BERT, developed in 2018 by Google AI Language as an NLP model uses MLM, which uses a somewhat convoluted approach of replacing 15% of the words with [MASK] and predicting them[5]. In practice, it is slightly more convoluted by masking 80%, replacing 10% with a random word, and keeping the other 10% as the same word to improve performance[5]. Whereas previous models used left to right or right to left Long Term Short Term Models (LSTMs) or left to right GPTs, this only learns context from the part of the sentence before the word, rather than the entire sentence itself, reducing performance [5]. For instance, “The woman went to the store and bought a ___ of shoes” requires context from both sides of the sentence to best predict the next word (pair). Additionally, BERT does Next Sentence Prediction (NSP) to understand the relationship between sentences, in which the model is fed 2 sentences at a time in which 50% of the time the second sentence comes after the first, the other 50% of the time the sentence is random, using [CLS] tokens to start the sentence and [SEP] tokens[5].



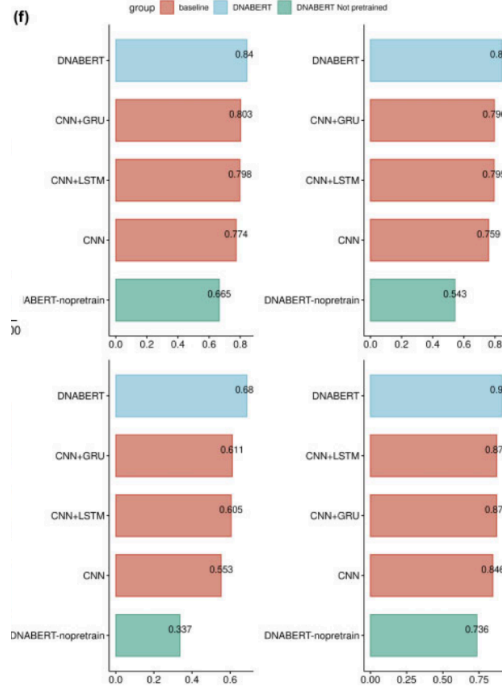
Since BERT is a transformer, it can be fine tuned for a specific task like answering a question, in which a single layer is added on top of the model, specific hyperparameters tuned, and trained[5].



2.2 DNABERT

While study of small, isolated segments of protein coding genes has been studied, much of DNA is noncoding and regulates expression of protein coding regions[6]. Multiple distant cis regulatory elements (CREs) may cooperate to use promoters in context-dependent use of promoters for special functional roles, such that distant sequence codes could be said to form semantic relationships[6]. Analyses of DNA have shown that it has many similarities to human language, especially in the noncoding regions, including alphabets, lexicons, grammar and phonetics[6]. Computational tools have used CNNs and RNNs to study the individual aspects of cis regulatory landscapes, but CNNs can't capture semantic dependency due to features being limited by filter size, while RNNs have issues from vanishing gradient and low efficiency due to having to sequentially process past states and compresses contextual information into a bottleneck with long sequences[6]. Additionally, these models need lots of data to run well, which is problematic in data scarce scenarios[6]. Since BERT, as a transformer has the ability to learn a great deal of data initially and then be fine tuned for a specific scenario with limited high quality data, as well as using the entire sequence as context instead of just the part before it like a GPT to better contextualize relationships between noncoding regions[6].

DNABERT uses k-mer tokens as input, with 4 versions using 4 different sized k-mer inputs (3, 4, 5, 6). While the same architecture is used as BERT, adaptations were made to the input by removing next sentence prediction, adjusting the sequence length to 10-510 length sequences, and predicting contiguous k-mer token to adapt to the DNA scenario. In the training done for DNABERT, it was pretrained from humane genome data to learn basic syntax, masks 15% of the tokens in the sequence, with cross entropy loss as the validation function, and is then fine tuned for promoters, TBFSS, and splice sites and compared to other models, with the AdamW optimizer used[6]. With pre training, DNABERT had superior performance compared to all other CNN, LSTM, and GRU implementations at each of these tasks, demonstrating the BERT based architecture was superior[6]. While the development of DNABERT was a milestone, it had limitations that hampered its effectiveness as a foundational model such as limited input size to 512 base pairs, worse performance compared to other transformer based models like NT, computationally expensive tuning time, as well as being trained on a limited dataset of only human genomic data. With these limitations, there was a clear and present need for a newer and more powerful model to improve upon DNABERT.



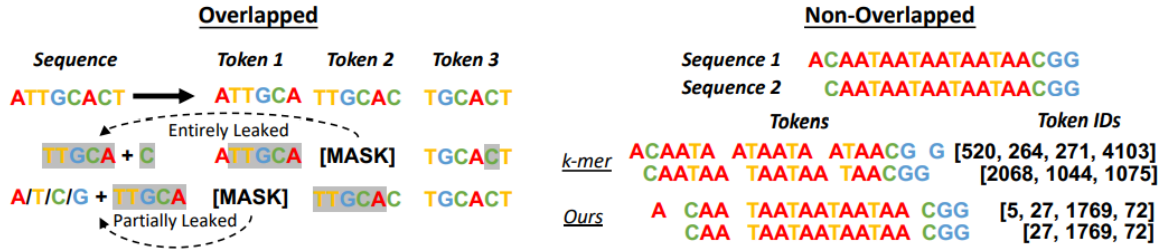
3 METHODOLOGY

Building off the initial success of DNABERT, DNABERT2, while retaining a BERT based architecture was a substantial overhaul to address issues with DNABERT and a competing transformer based model Nucleotide Transformer (NT). With improvements in tokenization, embeddings, attention calculation, training dataset size and diversity, as well as fine tuning optimization, DNABERT2 was able to achieve superior performance and allow for considerably larger input size limits for fine tuning while reducing model size, training time and fine tuning time.

3.1 Improvement Methods

Both DNABERT and NT use k-mers as their method of tokenization, with DNABERT using overlapped k-mers and NT using non-overlapped k-mers[7]. Overlapped k-mers suffer from total or partial leakage of k-mers from the tokens left and right of the masked token, such that the model can correctly piece together the token using the surrounding tokens instead of having to infer based on statistical patterns what the sequence in the mask should be[7]. Furthermore, given that an input sequence of length L consists of $L - k + 1$ tokens of length k , there is considerable redundancy and a length very close to the original length of the sequence, which considerably lowers computational efficiency since transformers have quadratic complexity, causing issues with model scalability[7]. While non overlapping k-mers address these issues, a

minor shift in the sequence results in a major adjustment to the tokenized output, creating just as big of an issue of sample inefficiency[7].



To address both of these issues DNABERT2 uses SentencePiece Byte Pair Encoding (BPE), which pairs frequent pairs of nucleotides and genome segments to form a vocabulary of variable-length tokens, preventing token leakage, reducing sequence length by around 5 times, and must guess not only the nucleotides, but also the number of nucleotides in a T5-style ‘replace spans of text’ objective that has been proven to result in superior performance compared to standard masked language modeling[7]. The BPE works by adding the most frequent character segment as a new word and replacing the segments with the new word until a target vocabulary size is reached. A vocabulary size of 2^{12} was determined to be the optimal vocabulary size based off of testing[7].

Iteration	Corpus	Vocabulary
0	AACGCACTATATA	{A,T,C,G}
1	AACGCACTATATA	{A,T,C,G,TA}
2	AACGCACTATATA	{A,T,C,G,TA,AC}
3	AACGCACTATATA	

Since the attention mechanism is permutation invariant, explicit positional information is needed[7]. Attention with Linear Biases (ALiBi) uses a fixed set of static, non-learned biases to each attention calculation since it does not have an input length restriction and extrapolates well to sequences longer than the training data. ALiBi defines q_i , the i -th query to an input sequence series of length L and K as:

$$A_i = \text{softmax}(qK + m * [-(i - 1), \dots, -2, -1, 0, -1, -2, \dots, -(L - 1 - i)])$$

where m is a head specific constant from a geometric sequence. ALiBi penalizes attention scores between key-query pairs as distances increase, with m as the penalty rate, allowing ALiBi to handle arbitrarily long sequences during fine tuning and inference despite being trained on shorter sequences.

DNABERT2 uses Flash Attention to compute the standard attention calculation by reducing reads and writes to High Bandwidth Memory (HBM)[7]. Finally, DNABERT2 uses Low Rank Adaptation to reduce computational and memory costs with ignorable performance sacrifice, by converting $\Delta W \in \mathbb{R}^{m \times n}$ to $\Delta W \rightarrow BA$ ($B \in \mathbb{R}^{m \times r}$, $A \in \mathbb{R}^{r \times n}$, $r \ll m$, $r \ll n$) to reduce the dimensions of ΔW from $m \times n$ to $r \times (m+n)$, as well as replacing the Relu activation function with GELU[7].

3.2 Data

DNABERT2 used the same 2.75B nucleotide human genome dataset used in DNABERT, as well as a 32.49B nucleotide base multi species dataset from 135 species[7] as a pretrain dataset, as well as a multi species test dataset Genome Understanding Evaluation (GUE) and GUE+ benchmark where DNABERT and NT models yielded moderate F1-scores between 0.3 and 0.8.

Species	Task	Num. Datasets	Num. Classes	Sequence Length
Human	Core Promoter Detection	3	2	70
	Transcription Factor Prediction	5	2	100
	Promoter Detection	3	2	300
	Splice Site Detection	1	3	400
Mouse	Transcription Factor Prediction	5	2	100
Yeast	Epigenetic Marks Prediction	10	2	500
Virus	Covid Variant Classification	1	9	1000

Table 1: Summarization of the Genome Understanding Evaluation (GUE) benchmark.

Species	Task	Num. Datasets	Num. Classes	Sequence Length
Human	Enhancer Promoter Interaction	6	2	5000
Fungi	Species Classification	1	25	5000
Virus	Species Classification	1	20	10000

Table 2: Summarization of the Genome Understanding Evaluation Plus (GUE⁺) benchmark.

3.3 Experiments and Results

Four variants of DNABERT (3mer, 4mer, 5mer, 6mer) and NT (NT-500M-human, NT-500M-1000g, NT-2500M-1000g, NT-2500M-multi) were chosen, in which human is from the hg38 project, 1000g 3202 high-coverage human genomes from the 1000 Genome project, and multi the genome of 850 different species. NT was too large for fine tuning, so only DNABERT and DNABERT2 were fine tuned. FLOPs (Floating Point Operations) were calculated on genome sequences of length 500, and an identical version of DNABERT2 (DNABERT2♦) was also pre-trained on the training sets of the GUE benchmark.

DNABERT2 achieved similar if not better performance while being 21x smaller and requiring 19x fewer FLOPs compared to NT, as well as one third the number of FLOPs as DNABERT. However, DNABERT achieved superior performance at the Core Promoter task since it had shorter input sequence lengths compared to DNABERT2, which because of compressing sequence lengths limits the amount of information retained and hinders informed decision making. Additionally, DNABERT2 did well at predicting 10000-bp sequences with only a few epochs of fine-tuning

Model	Params. ↓	FLOPs ↓	Trn. Tokens	Num. Top-2 ↑	Ave. Scores ↑
DNABERT (3-mer)	86M	3.27	122B	2 0	61.62
DNABERT (4-mer)	86M	3.26	122B	0 1	61.14
DNABERT (5-mer)	87M	3.26	122B	0 1	60.05
DNABERT (6-mer)	89M	3.25	122B	0 1	60.51
NT-500M-human	480M	3.19	50B	0 0	55.43
NT-500M-1000g	480M	3.19	50B	0 1	58.23
NT-2500M-1000g	2537M	19.44	300B	0 1	61.41
NT-2500M-multi	2537M	19.44	300B	7 9	<u>66.93</u>
DNABERT-2	117M	1.00	262B	8 4	66.80
DNABERT-2♦	117M	1.00	263B	11 10	67.77

4 DATA

To fine-tune DNABERT2 for DNA sequence error correction, we generated a robust dataset simulating realistic sequence errors. Below, we detail the process of data generation and preparation, in a manner that ensures reproducibility and clarity.

4.1 Reference Genome and Sequence Generation

We used a T2T (Telomere-to-Telomere) reference genome file for chromosome 21 that was in FASTA format (.fna) as the basis for sequence generation. The T2T genome is highly accurate and complete, making it an ideal reference for generating synthetic sequencing reads with understood error profiles.

Synthetic reads were generated using the wgism tool, which facilitates the creation of simulated sequencing reads. A total of 3.2 million sequences were produced, each with a length of 150 base pairs (bp). The tool was configured to introduce the following error probabilities at each base:

- Insertion/Deletion (Indel) Errors: 10% chance per base.
- Substitution Errors: 1% chance per base.

This sequence generation process yielded paired sequences:

1. Correct sequences: Representing the original reference sequences without errors.
2. Incorrect sequences: Corresponding sequences with simulated errors introduced as per the specified probabilities above.

The output from this step was saved in FASTQ format, capturing both the sequences and their quality scores.

4.2 Label Assignment for Error Correction

To prepare this generated data for fine-tuning DNABERT2, we compared the “correct” and “incorrect” (error-prone) sequences base by base. Labels were assigned to each base to indicate whether the base was correct or contained an error:

- “1”: The base is correct (no error).
- “0”: The base is incorrect (contains an insertion, deletion, or substitution).

Each sequence and its corresponding labels were formatted as follows:

- Sequence: A string of 150 bases (e.g., ATCTAGCT).
- Labels: A comma-separated string of binary values corresponding to each base in the sequence (e.g., "1, 1, 0, 1, 1, 1, 1, 0").

The sequence-label pairs were outputted in a structured CSV file following the schema shown:

```
sequence, label  
ACGTCA, "1,0,1,1,0,1"
```

This format enabled straightforward processing during model training and evaluation.

4.3 Data Splitting

The complete set of 3.2 million sequences was divided into training, validation, and testing subsets with an 80/10/10 split, ensuring sufficient data for each stage of model development. The split was performed randomly while maintaining balanced error distribution across the subsets to avoid potential biases.

- Training Set (80%): Used for model training.
- Validation Set (10%): Used to tune hyperparameters and monitor model performance throughout the training.
- Test Set (10%): Reserved for the final evaluation of the model's performance on unseen data.

This data generation and preparation pipeline ensured a high-quality dataset for fine-tuning DNABERT2. By simulating realistic sequencing errors and maintaining balanced, labeled, datasets we created a robust framework for training a model capable of accurate DNA sequence error correction.

5 MODIFICATIONS

DNABERT2 is inherently designed for binary classification tasks, such as promoter site detection and transcription factor binding prediction (Zhou et al., 2024). However, adapting the model for our multi-label classification task, where each base in a DNA sequence is assigned a label independently, required significant modifications to both its architecture and data processing pipeline. Below we detail the changes made and the challenges addressed to enable DNABERT2 to excel in this fundamentally different problem domain.

5.1 Architectural Modifications

1. Activation Function

- In DNABERT2’s original implementation, a softmax activation function was used in the output layer (Zhou et al., 2024). Softmax calculates probabilities for each output by normalizing them relative to one another, ensuring the probabilities sum to 1. This is effective for binary or multi-class classification tasks where one label is assigned per instance.
- For multi-label classification, however, we replaced the softmax activation function with a sigmoid activation function. Sigmoid independently computes the probability of each label being present, allowing multiple labels to be active simultaneously. This change ensures that each base’s label is determined independently, aligning with the requirements of our task.

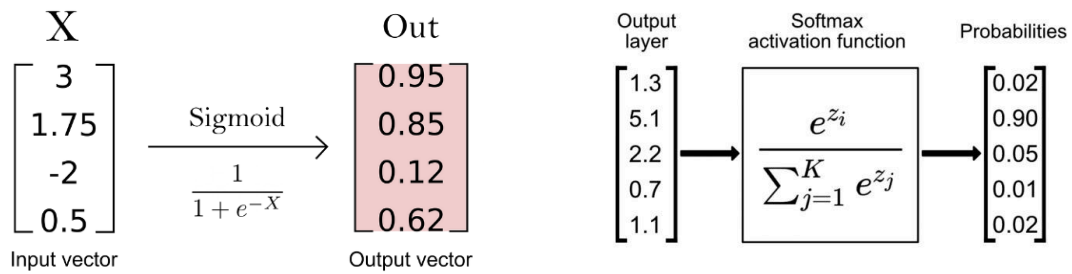


Figure X. Illustrates the differences between applying the sigmoid function versus the softmax function. Sigmoid computes the inputs independently, while softmax does not.

2. Multi-Label Encoding

- The labels for our task, as stated previously, indicate whether each base in a 150bp sequence is correct “1” or incorrect “0”. To adapt this for multi-label classification, we converted the labels into multi-hot encoded vectors, where each position in the vector corresponds to a base in the sequence.
- For example, a sequence with errors at the 3rd and 7th positions would be encoded as: “1, 1, 0, 1, 1, 1, 0, 1, 1”.
- These multi-hot encoded vectors were then converted into tensors, ensuring compatibility with PyTorch’s processing pipeline and allowing efficient computation during training.

3. Output Dimension Alignment

- The output layer’s dimension was explicitly set to 150, corresponding to the length of each input sequence. This ensured that the model produced a prediction for each base in the sequence, aligning directly with the task requirements.

5.2 Optimization Challenges and Fixes

While implementing these modifications, we encountered a critical issue related to the version of Triton being used, a Python library designed for efficient GPU computations. Triton was utilized for implementing flash attention to optimize the training process. However, we experienced errors due to improper handling of matrix transpositions in the code used for dot product calculations.

- **Error Description:** The original implementation contained the following line:

```
qk += tl.dot(q, k, trans_b=True)
```

This code attempted to perform a matrix multiplication with a transposed version of the second matrix, k. However, improper handling of the transposition caused runtime errors and inefficient computation.

- **Solution:** To resolve this, we explicitly applied a transposition operation to k before performing the dot product:

```
qk += tl.dot(q, tl.trans(k))
```

This modification ensured correct matrix alignment and significantly improved the stability and efficiency of the computation, enabling smoother and faster training.

These architectural and computational changes enabled DNABERT2 to handle the complexities of multi-label classification effectively. The use of sigmoid activation allowed independent estimation for each base, while the multi-hot encoded labels ensured that the data was processed in a manner consistent with the task requirements. By addressing the issues with Triton's matrix operations, we further optimized the training pipeline, leveraging efficient GPU computations to handle the computational demands of large-scale DNA sequence error correction tasks. Together, these modifications allowed DNABERT2 to be successfully adapted for this novel application, demonstrating versatility and capability in addressing diverse sequence-based tasks.

6 EXPERIMENTS

In this section, we provide detailed information relating to the training, evaluation, and performance of our fine-tuned model compared to base DNABERT2.

6.1 Training

The training phase utilized 80% of the 3.2 million sequences generated, corresponding to approximately 2.5 million 150bp sequences and their associated labels. These sequences were used to fine-tune the pre-trained DNABERT2 model, adapting it to the specific task of DNA sequence error correction. The training configuration followed best practices from the DNABERT2 documentation while incorporating adjustments to optimize performance for our multi-label classification problem:

- **Learning Rate:** The learning rate was set to $3e-5$, consistent with the recommended value for DNABERT2 fine-tuning.
- **Batch Sizes:** The training batch size was increased to 12, and the evaluation batch size was set to 32. To simulate larger effective batch sizes, we employed gradient accumulation steps with a factor of 2.
- **Epochs:** Training was conducted over three epochs to mitigate the risk of overfitting while allowing the model sufficient time to adjust to the task.
- **Regularization:** A weight decay of 0.01 was applied to discourage overfitting and improve generalization.
- **Mixed Precision Training:** This technique was used to optimize performance, reducing memory usage and increasing computational efficiency.
- **Hardware:** Training was performed on a single NVIDIA A100 80GB GPU. The process took approximately two days, with a training speed of about 25 iterations (batches) per second.

6.2 Evaluation

The evaluation of the model was conducted using Binary Cross-Entropy with Logits Loss (BCEWithLogitsLoss) function. This loss is particularly well-suited for multi-label classification tasks. Unlike traditional binary cross-entropy loss, BCEWithLogitsLoss combines a sigmoid activation function and binary cross-entropy into a single function, offering numerical stability and computational efficiency. It measures the discrepancy between the predicted probabilities and the ground truth labels, guiding the model to minimize this difference during training. Model performance was monitored at the end of each epoch with this loss function.

Performance was measured at the end of training using the test dataset and included the following evaluation metrics:

- **Accuracy:** Measures the proportion of correctly predicted labels across all bases.
- **Precision:** Reflects the proportion of correctly predicted positive labels (correct bases) out of all predicted positives.
- **Recall:** Indicates the proportion of actual positive labels (correct bases) that were correctly identified.
- **F1 Score:** Harmonic mean of precision and recall, providing a balanced measure of model performance.

During evaluation, the sigmoid function was applied to the model's outputs to convert logits into probabilities. A threshold of 0.5 was then used to determine binary predictions. Probabilities greater than 0.5 were classified as correct bases "1", while probabilities less than or equal to 0.5 were classified as incorrect bases "0".

6.3 Results

The results of the fine-tuned model were compared to the performance of the base DNABERT2 model on the test dataset. A summary of the performance metrics is presented in the table below:

Metric	Base DNABERT2	Fine-Tuned DNABERT2
Accuracy	0.000	0.222
Precision	0.990	0.990
Recall	0.486	1.0
F1 Score	0.651	0.995

The fine-tuned model demonstrated a substantial improvement in task specific performance:

1. **Accuracy:** The accuracy increased from 0 to 0.22, reflecting a significant enhancement in the model's ability to make correct predictions across the sequence bases.
2. **Precision:** Both models maintained high precision (0.99), indicating that false positives were minimal for both the base and fine-tuned models.
3. **Recall:** The fine-tuned model achieved perfect recall, addressing the base model's major limitation in identifying all correct bases.
4. **F1 Score:** The fine-tuned model's F1 score increased to 0.99, a marked improvement from the base model's score of 0.651, demonstrating a balanced enhancement in both precision and recall.

These results underscore the effectiveness of the fine-tuning process and the architectural modifications made to DNABERT2. By addressing the model's major weakness, particularly in recall, and maintaining its strength, the fine-tuned model offers a robust solution to the DNA sequence error correction task.

7 CONCLUSION

The advent of large language models (LLMs) has revolutionized the ability to learn and generalize complex patterns from both upstream and downstream contexts. This capability is particularly important for tasks such as correction errors in DNA sequences, where context from surrounding bases plays a vital role in determining the accuracy of predictions. DNA sequencing has become a pivotal tool in modern biology and medicine, facilitating breakthroughs in genomics, diagnostics, and personalized medicine. However, despite significant advancements in sequencing technologies, relatively high error rates persist in certain sequencing methods, necessitating robust post-sequencing error correction strategies.

In this study, we fine-tuned DNABERT2, a pre-trained DNA-specific language model, to address the challenge of DNA sequence error correction. Our modifications adapted DNABERT2 from its original binary classification framework to a multi-label classification task, enabling it to process and predict the correctness of individual bases in a sequence. While our fine-tuned model demonstrated a substantial improvement in performance compared to the base DNABERT2 model, particularly with respect to recall, this result was anticipated, given that DNABERT2 was not originally designed for this error correction task. Nevertheless, our work underscores the potential of LLMs to excel in highly specialized tasks when appropriately fine-tuned.

Although the results are promising, our model is far from being a deployable solution for real-world DNA error correction. Its current limitations necessitate further research and development to improve accuracy, generalizability, and robustness. Future directions for enhancing the model include fine-tuning on a larger and more diverse dataset, incorporating real sequencing data instead of synthetic data, and leveraging hyperparameter optimization to refine model performance. Additionally, DNABERT2's training on data from 135 species (Zhou et al., 2024), compared to our fine-tuning on only human data, raises questions about the impact of species diversity on model predictions, which warrants further investigation.

To comprehensively evaluate our model, comparisons with simpler error correction techniques should be conducted to establish benchmarks and validate the utility of our approach. Another critical future goal would be to extend the model's capabilities to not only detect incorrect bases but also suggest accurate corrections. This functionality would mark a significant step toward practical applications in sequencing error correction pipelines.

In conclusion, our study highlights the adaptability and potential of LLMs like DNABERT2 for addressing domain-specific challenges such as DNA sequence error correction. While our fine-tuned model serves as a foundational demonstration of what is achievable, significant work remains to advance this approach into a robust, deployable solution. By addressing current limitations and exploring the outlined avenues for improvement, we anticipate that future iterations of this model could play a transformative role in enhancing the accuracy and reliability of DNA sequencing technologies.

REFERENCES

[1] *DNA sequencing methods: From Past to Present.* | Read by QXMD. (n.d.). Read by QxMD. <https://read.qxmd.com/read/36655445/dna-sequencing-methods-from-past-to-present>

- [2] Mohammadi, M. M., & Bavi, O. (2021). DNA sequencing: an overview of solid-state and biological nanopore-based methods. *Biophysical Reviews*, 14(1), 99–110. <https://doi.org/10.1007/s12551-021-00857-y>
- [3] Satam H, Joshi K, Mangrolia U, Waghoo S, Zaidi G, Rawool S, Thakare RP, Banday S, Mishra AK, Das G, et al. Next-Generation Sequencing Technology: Current Trends and Advancements. *Biology*. 2023; 12(7):997. <https://doi.org/10.3390/biology12070997>
- [4] Benegas, Gonzalo, et al. “Genomic Language Models: Opportunities and Challenges”. ArXiv.org, 2024, arxiv.org/abs/2407.11435. Accessed 14 Dec 2024.
- [5] BERT: Devlin, Jacob, et al. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. 24 May 2019.
- [6] DNABERT: Yanrong Ji, Zhihan Zhou, Han Liu, Ramana V Davuluri, DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome, *Bioinformatics*, Volume 37, Issue 15, August 2021, Pages 2112–2120, <https://doi.org/10.1093/bioinformatics/btab083>.
- [7] DNABERT2: Zhou, Zhihan, et al. “DNABERT-2: Efficient Foundation Model and Benchmark for Multi-Species Genome.” ArXiv.org, 2023, arxiv.org/abs/2306.15006. Accessed 11 Dec. 2024
- [8] Ligeti, B., Szepesi-Nagy, I., Bodnár, B., Ligeti-Nagy, N., & Juhász, J. (2024). ProkBERT family: genomic language models for microbiome applications. *Frontiers in Microbiology*, 14. <https://doi.org/10.3389/fmicb.2023.1331233>